Sprint 3 Report

Kaila · Database Backend · SOEN 341 CareerNet

1. Summary

In Sprint 3 I owned the database side of notifications and part of the AI resume feedback feature. I merged the two login helpers into one, built the notifications table, migration and notification service, a demo data script, and cleaned up the requirements and README. For the AI feature, I wrote the resume text reader and a per-user rate limiter for AI requests. I added **50 automated tests** across these tasks and drafted the US-5 acceptance test.

2. Tasks

Task	What	Issue	Pull requests
KA-9	One login helper	—	#122
KA-6	Notifications table, interview date, migration	#155	#123, #128 (revert), #130
KA-7	Notification service	#156	#124, #131
KA-8	Demo data script	#160	#125
KA-10	Requirements and README cleanup	—	#126
KA-12	Read text from PDF and Word resumes	#163	#132, #133
KA-13	Limit AI requests per user per hour	#168	#173
AT-5	US-5 notifications acceptance test	#161	—
KA-11	Personal report and logs	—	#127, this PR

3. What I Did

KA-9 · One login helper

Made `App/core/dependencies.py` the only login helper, so every protected route answers 401 “Not logged in.” the same way. Investigated first (path C then A), pointed four routers at it and deleted `deps.py`. 17 tests.

KA-6 · Notifications table, interview date and migration

Notifications table with delete rules and a unique rule on user and event key, `applications.interview_at`, and migration `1dae1f3df735` applied to Neon. 5 tests. After it was reverted in #128, I restored it by reverting the revert (#130).

KA-7 · Notification service

Create, notify, list, count, mark-read and interview-reminder functions in `App/crud/notifications.py`, used by Lamees's LA-7 endpoints (#189) and Youssef's YO-7. The functions never commit, so a

notification is saved together with the change that caused it. 15 tests.

KA-8 · Demo data script

`scripts/seed_demo_data.py` adds demo accounts, 12 jobs, 12 applications in all five statuses and notifications. Asks for YES first, does nothing if run twice, commits once. Loaded on Neon.

KA-10 · Requirements and README cleanup

One line per package in `requirements.txt` (kept httpx2 after a clean-environment test showed the test client needs it) and a README that lists only real features, with API and testing sections.

KA-12 · Read text from PDF and Word resumes

Libraries in their own PR (#132). `extract_resume_text` in `App/services/resume_text.py` reads PDFs and .docx files (including tables), cleans and cuts the text at 12,000 characters, and gives safe errors for scanned, old .doc and broken files. Shared sample-file builders in `tests/resume_samples.py`. 9 tests.

KA-13 · Limit AI requests per user per hour

In-memory limiter in `App/core/rate_limit.py`: 5 feedback requests per user per hour. 4 tests. The feedback endpoint checks it before doing any work and answers 429 when the limit is reached.

AT-5 · US-5 acceptance test

Wrote the Given / When / Then details, manual steps and the automated tests that cover each part of the notifications story.

4. Time Spent

Date	Work	Hours
Nov 4	Setup on a second computer, KA-9, KA-6, KA-7, KA-8, KA-10, acceptance test drafts	3.25
Nov 5	Restoring KA-6 and moving KA-7 into main	0.75
Nov 7	KA-12	1.0
Nov 8	KA-8 into main, KA-12 conflict fix, issues, KA-13	1.25
Nov 12–14	Labels, merging LA-7, report and logs	1.75
Total		**≈ 8**

5. What I Can Explain in the Lab

- **Login helper (KA-9):** why there were two helpers with different messages, why `HTTPBearer(auto_error=False)`, and how the tests cover missing, wrong, bad and expired tokens and deleted users.
- **Notifications table (KA-6):** what ON DELETE CASCADE and SET NULL do here, why the unique rule stops duplicate reminders, and why I wrote the migration by hand and checked it with `alembic check`.
- **Migrations on a shared database:** previewing with `alembic upgrade head --sql`, running it on Neon, and why main and the database must have the same migration files.

- **Notification service (KA-7):** why the functions flush but never commit, and how reminders work without a background timer.
- **Reverting and restoring:** why stacked pull requests broke, why merging the old branch again adds nothing, and how reverting the revert brought KA-6 back.
- **Resume text reader (KA-12):** what it can and can't read, why errors never contain resume text, and how the tests build real PDF and Word files.
- **Rate limiter (KA-13):** how request times are kept per user, and why passing the time in makes a one-hour rule testable instantly.
- **Demo data (KA-8):** why it asks for YES, stops if the demo accounts exist, and commits once.

6. AI Evidence

AI tool: Claude. Every use is also in my AI usage log (`AI_Doc/AI Usage Log - Kaila.pdf`, Tasks 22 to 38). For each use: the prompt, what the AI helped with, my decision, and what I learned. How I checked each one is in the decision and reflection.

Setup and KA-9: One Login Helper

Task 22 - Setting Up the Project on a Second Computer

Prompt: "I'm on a different computer and want to install requirements.txt before starting KA-9, but the venv won't activate. What's the right order to set this up, and what should I check before committing from this machine?"

Purpose: Environment setup

How AI helped:

- Explained that my first install went into the computer's main Python because the venv didn't exist yet, and walked me through creating .venv.
- Explained the PowerShell execution policy and how to allow scripts for my user only.
- Pointed out that the folder path had a different username than last time, so I should check which name and email my commits would use.
- Noticed that my Git email was already my GitHub no-reply address, which links commits to my account, after I had replaced it with my Gmail.

Decision: Modified. I switched my Git email back to the GitHub no-reply address.

Reflection: Individual contribution is read from the commit history, so a wrong Git identity could have made my work look like someone else's. Setting up a new machine is a good moment to check that, not something to skip.

Task 23 - Looking Before Changing: Which Path for KA-9?

Prompt: "Before I change either login helper, can you help me read the step 0 results and figure out which path actually fits our code?"

Purpose: Code investigation

How AI helped:

- Gave me PowerShell versions of the guide's step 0 commands (git grep instead of grep).
- Read my results with me: four routers used deps.py, two used dependencies.py, and the only frontend match for "Not logged in" was a comment.
- Explained that no test checks the message text, only the status code, so no existing test would need to change.
- Recommended path C then A, because the missing-header case was the only real difference between the two helpers.

Decision: Accepted, after checking the grep output myself.

Reflection: The investigation took less time than the change, and it decided everything. Knowing that nothing depended on the message text made the change much less risky than it first looked.

Task 24 - Merging the Helpers and Proving Nothing Broke

Prompt: "Can you help me make dependencies.py the only helper and write tests that prove every bad login gets the same answer? And did my change cause the app to crash?"

Purpose: Implementation, testing and debugging

How AI helped:

- Rewrote get_current_user in dependencies.py with HTTPBearer(auto_error=False), so a missing header gets our own "Not logged in." instead of the library's message.
- Wrote 17 tests that try three bad headers, an expired token and a deleted account on three different routes, plus the 403 for a job seeker on a recruiter route.
- Explained that the crash came from a missing .env on this computer, not from my code, and gave me a way to check the app starts using a throwaway local database.
- Corrected its own test count when I got 124 instead of the 125 it predicted.

Decision: Accepted. 124 tests passed and the app started with no import errors.

Reflection: Testing every combination is what proves the messages are really the same everywhere. I also learned to read an error carefully before assuming my change caused it: the traceback pointed at DATABASE_URL, not at the login helper.

KA-6: Notifications Table and Migration

Task 25 - Designing the Notifications Table

Prompt: "How should the notifications table behave when a user or application is deleted, and how does the event key stop duplicate reminders? I can't connect to the database from this computer yet, so how do I write the migration?"

Purpose: Database design and migration

How AI helped:

- Explained the delete rules: CASCADE removes a user's notifications with the user, while SET NULL keeps a notification but clears its link if the application is deleted.
- Explained that a unique rule on user and event key lets the database itself refuse a duplicate reminder, while notifications with no event key are still allowed.
- Wrote the migration by hand, since autogenerate needs a database connection, and suggested alembic check afterwards to confirm it matches the models.
- Pointed out that SQLite only follows delete rules when foreign keys are switched on, so the delete test turns them on and off again.

Decision: Accepted. I asked whether the docstring at the top of the migration was necessary, and kept it to match the other migrations.

Reflection: This was the same kind of thinking as the applications table in SA-4: what happens to real data when something is deleted. Writing the migration by hand also made me read every line of it instead of trusting a generated file.

Task 26 - Connecting to Neon from a New Computer

Prompt: "My .env is empty and I don't remember where the secret key is. Is it the same for everyone, and why is Alembic using SQLite instead of Neon?"

Purpose: Configuration and debugging

How AI helped:

- Found in our README that each teammate makes their own JWT secret, and that the values I remembered from config.py and conftest.py are only fallbacks and test values.
- Gave me one command to write the .env and generate the secret, and to check that Git ignores the file.
- Explained that the temporary SQLite setting from the earlier startup check was still active in my terminal, because Ctrl+C stopped the cleanup lines from running.
- Reminded me that the database link contains the password and shouldn't be pasted into chats or commits.

Decision: Accepted

Reflection: Settings in the terminal take priority over the .env file, which is why Alembic was talking to the wrong database. Checking alembic current before migrating caught it, which is exactly why that check comes first.

Task 27 - Running the Migration on the Shared Database

Prompt: "The preview shows every migration from the start. Will it rebuild our tables? And if it's already in the database, why do I still need a pull request?"

Purpose: Migration review on a shared database

How AI helped:

- Explained that --sql mode doesn't look at the database, so it prints the whole chain, while the real upgrade only runs what is missing.
- Pointed me to the part that mattered: only the notifications table, its index and interview_at, with nothing dropped.
- Explained that the database and the code are separate: the table now exists on Neon, but the model and migration files only existed on my computer, so teammates couldn't use them and Alembic would error for them.

Decision: Accepted. I told the team before running it, and afterwards alembic current showed 1dae1f3df735 and alembic check found no differences.

Reflection: I'm glad I asked before running it. A migration has two halves, the change in the database and the file in the repository, and the team needs both.

KA-7: Notification Service

Task 28 - Notification Functions Other People Will Build On

Prompt: "Lamees and Youssef will call these functions, so why shouldn't they commit on their own? And how can reminders work without a background timer?"

Purpose: Service design and testing

How AI helped:

- Explained the no-commit rule: the caller commits once, so a notification is only saved if the change that caused it is saved too.
- Explained that reminders are created when the user opens their notifications, and that putting the interview time in the event key means a rescheduled interview gets a new reminder.
- Pointed out that SQLite returns dates without a time zone, so those are treated as UTC before comparing.
- Wrote 15 tests, including that a rollback removes an unsaved notification and that a stranger can't mark someone else's notification as read.

Decision: Accepted. 127 tests passed, and I posted the function names for Lamees and Youssef before the merge.

Reflection: Writing code that others depend on means thinking about how they'll use it, not just whether it works. Opening the pull request early lets them start from the function names instead of waiting for me.

Task 29 - Writing Acceptance Tests Before the Feature Exists

Prompt: "The notification endpoints and screens aren't built yet. How do I write US-5 acceptance tests that are still useful now and can be run later?"

Purpose: Acceptance testing

How AI helped:

- Explained the Given / When / Then format from our Team Guide and that each scenario is its own issue.
- Drafted the three US-5 scenarios: a status change notifies the job seeker, a new application notifies the recruiter, and an interview within 24 hours creates a reminder.
- Suggested pointing each draft at my existing service tests for now, and marking the app-level run as waiting for LA-7, LA-8, YO-7 and DA-7.

Decision: Kept the drafts, but decided to post them once Sayf creates the US-5 story issue, so they can link to it.

Reflection: Writing the scenarios early made me think about the feature from the user's side, not just the database side. It also showed which parts depend on my teammates' tasks.

KA-8: Demo Data Script

Task 30 - A Demo Script That Can't Hurt the Shared Database

Prompt: "This script writes to Neon, which the whole team uses. How do I make sure it can't run by accident or add everything twice, and that the demo actually shows every status and a reminder?"

Purpose: Script design and test data

How AI helped:

- Followed the style of my Sprint 2 seed script and reused my KA-7 functions to create the notifications.
- Added a check that stops if the demo recruiter already exists, and a prompt that shows the database host and only continues if I type YES.
- Spread 12 applications over the last 30 days with all five statuses, and put one interview less than 24 hours away so a reminder appears.
- Used @example.com emails, because our login validation rejects reserved domains like .test.
- Replaced the plain job list with more interesting companies when I asked for cooler jobs.

Decision: Modified. I asked for a more interesting set of jobs, then accepted it.

Reflection: The YES prompt did its job right away: I typed "yes" in lowercase and it cancelled, which is exactly the kind of mistake it's there to catch. Running it twice and seeing "Nothing was added" proved it's safe for the team to use.

Task 31 - Why Some Pull Requests Weren't Being Checked

Prompt: "KA-9 and KA-6 show the CI checks, but KA-7 and KA-8 don't. Why, and how do I fix it without breaking anything?"

Purpose: CI and GitHub workflow

How AI helped:

- Read our CI workflow and found it only runs on pull requests into main, while KA-7 and KA-8 were based on my KA-6 and KA-7 branches.
- Gave two options: retarget the pull requests to main now, or wait until the earlier ones are merged.
- Advised not to change the CI file, since it belongs to Sayf's task.

Decision: Accepted

Reflection: Stacking pull requests on each other kept the diffs small, but it skipped the CI checks. Knowing when CI runs helps me make sure every change is actually tested before it's merged.

KA-10: Requirements and README Cleanup

Task 32 - Removing a Package That Turned Out to Be Needed

Prompt: "The guide says to remove httpx2 if nothing imports it, and grep finds nothing. But the clean-environment test shows a warning telling me to install httpx2. Which one should we keep?"

Purpose: Dependency cleanup

How AI helped:

- Found that pytest was listed twice and that grep shows no direct use of httpx2 in our code.
- Gave me commands to prove the file is complete in a brand-new environment on Windows.
- Read the warning with me: the test client from FastAPI and Starlette now expects httpx2, so httpx was the deprecated one.
- Had me test the reverse change in a second clean environment, which passed with no warning.
- Rewrote the README Features list so it only claims what exists, and added the API and Running Tests sections.

Decision: Modified. I kept httpx2 and removed httpx, the opposite of the guide's suggestion, because the clean test showed it.

Reflection: Grep only finds what our own code imports, not what a library uses behind the scenes. Testing in a clean environment caught something a quick search missed, and reading the warning instead of ignoring it gave a cleaner result.

Getting KA-6 and KA-7 into Main

Task 33 - Fixing Pull Requests That Never Reached Main

Prompt: "KA-7 was merged into my KA-6 branch, and KA-6 was reverted from main. Do I make a new KA-6 pull request, and how do I get both back into main without breaking anything?"

Purpose: Git and pull request recovery

How AI helped:

- Checked main and found that KA-6 had been merged and then reverted, and that KA-7 had been merged into the old KA-6 branch, so neither was in main.
- Pointed out that main was now missing the migration Neon is on, so Alembic would fail for everyone.
- Explained that merging the old branch again would add nothing, because Git sees it as already merged, and that reverting the revert brings the same changes back.

- Tested the revert of the revert on the current main before I ran it, and suggested asking Sayf why it was reverted first.
- Helped me finish a merge I left paused when I closed the editor.

Decision: Accepted, after Sayf confirmed it was fine to bring KA-6 back.

Reflection: Stacking pull requests on each other kept them small, but one merge into the wrong branch broke the whole chain. I first assumed KA-6 was safely in main, and checking the real history is what showed it wasn't. From now on I open every pull request straight into main.

KA-12: Read Text from PDF and Word Resumes

Task 34 - Adding Libraries the Whole Team Can Use

Prompt: "The guide wants the libraries in their own small pull request first. How do I add them with exact versions like the rest of requirements.txt, and will it clash with my KA-10 pull request?"

Purpose: Dependency management

How AI helped:

- Gave me a command that adds the three libraries with the exact versions I installed, to match the pinned style of the file.
- Explained what each library is for: pypdf for PDFs, python-docx for Word files, and fpdf2 only for building test PDFs.
- Warned that KA-10 also changes requirements.txt, so whichever is merged second needs a small conflict fix.

Decision: Accepted

Reflection: Putting the libraries in their own tiny pull request means Lamees and Youssef can start using them before my main KA-12 work is finished.

Task 35 - Reading Any Resume, Not Just One Format

Prompt: "Will this read only this specific resume format, or any? And if someone uploads a doc, does it still work, or do I need to change something?"

Purpose: Understanding the feature's limits

How AI helped:

- Explained that this step only pulls text out of the file, and the AI feedback comes later in Lamees's endpoint, so any resume layout works.
- Listed what can't be read: scanned PDFs, old .doc files, and Word text in headers, footers or text boxes. Two-column PDFs can come out in a mixed order.
- Explained the difference between .docx (works), old .doc (clear error message) and a Google Docs link (download it as .docx or PDF first).
- Said no change is needed for KA-12, since the guide only asks for PDF and .docx, and suggested raising .doc support as a possible Sprint 4 idea.
- Wrote extract_resume_text, the sample-file builders in tests/resume_samples.py and 9 tests.

Decision: Accepted. I kept the guide's scope and will mention .doc support to the team.

Reflection: Knowing what the reader can't handle matters as much as what it can. The error messages tell users exactly what to upload instead, and they never repeat anything from the resume.

Task 36 - Stopping a Pull Request from Merging into the Wrong Branch

Prompt: "Why is KA-8 going into KA-7? We don't want that. Are any of my other pull requests doing this, and how do I fix the conflict Sayf sent me on KA-12?"

Purpose: Git and pull request recovery

How AI helped:

- Checked which branch each of my open pull requests targets, and found that only KA-8 still pointed at the KA-7 branch.
- Had me point KA-8 at main and delete the merged branches, so nothing could target them again.
- Found that my updated log PDFs had been committed on the KA-12 branch by mistake, which caused the conflict after my logs pull request was merged.
- Tested the fix on a copy first, then had me merge main into KA-12 and keep main's PDFs, so the pull request only has the KA-12 code.

Decision: Accepted. I also decided that every pull request from now on goes straight into main.

Reflection: Each problem came from a branch being somewhere I didn't expect. Checking the real target branch and the files a pull request changes, before merging or fixing anything, is what made each fix safe.

KA-13: Limit AI Requests per User per Hour

Task 37 - A Limit That Doesn't Depend on the Clock

Prompt: "How do I limit each user to 5 AI feedback requests per hour, and test it without waiting an hour? Lamees's endpoint isn't merged yet, so what can I do now?"

Purpose: Implementation and testing

How AI helped:

- Wrote a small in-memory limiter that remembers each user's recent request times and forgets ones older than an hour.
- Added an optional now parameter, so the tests use a fixed time instead of the real clock.
- Checked GitHub and found that LA-10 had not started yet, so the endpoint part had to wait.
- Explained why the pull request should be a draft: it stops anyone from merging a limiter that nothing uses yet.

Decision: Accepted. I opened it as a draft and asked the team to tell me when LA-10 is merged.

Reflection: Passing the time in as a parameter made the hour-long rule testable in milliseconds. Splitting the work around a teammate's task let me finish my part without stacking on her branch.

Labels and Wrap-up

Task 38 - Labeling Every Pull Request at Once

Prompt: "All my pull requests, open and closed, should have labels like sprint-3, P1 and backend. Can I do that from the terminal instead of clicking through each one?"

Purpose: Project management on GitHub

How AI helped:

- Mapped each of my pull requests to its sprint, priority and area labels, using the priorities in my guide.
- Helped me install and log in to the GitHub CLI on my second computer, including reloading the PATH so the terminal found it.

- Found why every edit first failed: in PowerShell, an ordered list read with a number like 122 is treated as a position, not a key, so no labels were passed. The fixed loop went through the list as pairs.

Decision: Accepted, after checking every label name with gh label list first.

Reflection: Labels are part of the project management grade, so it was worth fixing them everywhere. When all twelve edits failed the same way, the cause had to be in my loop, not the labels.